GNNs vs. Proven Approximation Algorithms: A Rigorous Empirical Audit on NP-Hard Problems

Alan Zhang

June 2026

Abstract

Graph Neural Networks (GNNs) have been proposed as learned solvers for NP-hard combinatorial optimization problems, yet their performance relative to classical algorithms with *proven approximation guarantees* remains poorly characterized. We conduct a rigorous empirical audit comparing GIN-based GNN solvers against classical algorithms on three canonical NP-hard problems: Maximum Cut, Travelling Salesman Problem, and Graph Coloring. For Max-Cut, the Goemans-Williamson SDP relaxation ($\alpha \geq 0.878$) dominates the GNN across all graph families, with the GNN winning only 1.8% of 600 instances. For TSP, NN+2-opt local search matches or outperforms the GNN+2-opt on all 90 Euclidean instances, with the GNN providing no advantage in initialization quality. For Graph Coloring, DSatur produces colorings using 3.4× fewer colors than the GNN, which wins 0% of 600 instances. We further show that spectral graph properties—particularly the Laplacian spectral radius, degree variance, and spectral gap—are the strongest predictors of GNN failure, yet the prediction task itself is at chance: balanced accuracy 0.52 and F1 0.12, with the raw accuracy (57% LR, 78% RF) no better than the 98.2% majority-class baseline. The GNN's primary advantage is inference speed (up to 60× faster than SDP), not solution quality. These results demonstrate that for practitioners, classical algorithms with mathematical guarantees remain the reliable baseline, and GNNs should be evaluated against them—not just against naive heuristics.

1 Introduction

Combinatorial optimization over graphs—finding optimal cuts, shortest tours, minimal colorings—is a cornerstone of both computer science theory and applied operations research. Many of these problems are NP-hard [5], meaning no polynomial-time algorithm can solve all instances exactly (unless P = NP). However, decades of work in approximation algorithms have produced remarkable results: polynomial-time algorithms with *provable* worst-case guarantees on solution quality [9].

Recent years have seen growing interest in using Graph Neural Networks (GNNs) to learn solvers for these problems [7, 1, 3]. The appeal is clear: GNNs can leverage structural patterns in graph data, amortize computation across instances, and potentially discover heuristics that exploit distributional regularity.

However, a troubling gap exists in the literature: GNN solvers are typically compared against simple baselines (random, greedy) rather than against the best classical algorithms with known guarantees. This creates an inflated impression of GNN effectiveness. A GNN that beats a random partition on Max-Cut is unremarkable; one that beats the Goemans-Williamson SDP relaxation [6] would be extraordinary.

Contributions. We provide a rigorous empirical audit comparing GNN solvers against the strongest classical algorithms on three problems:

1. **Maximum Cut**: GNN (GIN + local search) vs. Goemans-Williamson SDP ($\alpha \geq 0.878$), spectral relaxation, and greedy, across 600 instances from 5 graph families.

2. **Travelling Salesman Problem**: GNN vs. Christofides ($\alpha \leq 1.5$) and 2-opt local search, on 90 Euclidean instances.

3. **Graph Coloring**: GNN vs. DSatur [2] and Welsh-Powell, across 600 instances from 5 graph families.

4. **Failure Prediction**: We train classifiers on spectral graph properties to predict when GNNs fail. The task is at chance—balanced accuracy 0.52, F1 0.12—because the GNN wins too rarely (1.8%) to learn a useful predictor; the raw accuracy (57% LR, 78% RF) is no better than the 98.2% majority-class baseline.

2 Background

2.1 Approximation Algorithms and Guarantees

An α-approximation algorithm for a maximization problem produces a solution of value $\geq \alpha \cdot \mathrm{OPT}$ in polynomial time. For minimization, $\leq \alpha \cdot \mathrm{OPT}$.

Max-Cut. Given a graph $G = (V, E)$, find a partition $(S, V \setminus S)$ maximizing the number of edges crossing the cut. The Goemans-Williamson algorithm [6] solves an SDP relaxation and rounds via random hyperplanes, achieving $\alpha \geq 0.878$. Under the Unique Games Conjecture, this is optimal.

TSP. Given n cities with pairwise distances satisfying the triangle inequality, find the shortest Hamiltonian tour. The Christofides-Serdyukov algorithm [4] computes an MST, adds a minimum-weight perfect matching on odd-degree vertices, and shortcuts the resulting Eulerian tour, achieving $\alpha \leq 1.5$. This stood as the best known approximation for nearly 50 years.

Graph Coloring. Given a graph, assign colors to vertices such that no adjacent vertices share a color, minimizing the number of colors. This problem is notoriously hard to approximate: for any $\epsilon > 0$, it is NP-hard to color a 3-colorable graph with $n^{1-\epsilon}$ colors. DSatur [2] is a widely-used heuristic that is optimal for bipartite graphs.

2.2 Graph Neural Networks for Combinatorial Optimization

GNNs learn to map graph-structured inputs to node or edge-level predictions through message-passing layers [10]. For combinatorial optimization, the typical approach is unsupervised: the GNN outputs soft assignments (e.g., partition probabilities for Max-Cut), trained with a loss that directly encodes the objective [7, 8].

We use the Graph Isomorphism Network (GIN) [10] throughout, as it is as expressive as the 1-WL test (the most expressive *standard* message-passing GNN; higher-order GNNs are strictly more expressive).

3 Experimental Setup

3.1 Graph Families

We test generalization by training on one family and evaluating on five:

- **Erdős-Rényi** $G(n, p)$: random edges with $p = 0.15$

- **Planted Partition**: stochastic block model with 2 communities
- **Random Regular**: d-regular graphs ($d = 3$)
- **Barabási-Albert**: preferential attachment ($m = 3$)
- **Watts-Strogatz**: small-world ($k = 4$, $p = 0.3$)

Graph sizes: $n \in \{20, 50, 100, 200\}$, with 30 instances per (family, size) pair, totaling 600 instances per problem.

3.2 GNN Architecture

All GNN solvers use a 5-layer GIN with 128-dimensional hidden states and batch normalization. For Max-Cut, the output is a per-node sigmoid probability; for Coloring, a softmax over 20 possible colors; for TSP, node embeddings scored by dot product. All GNN outputs are refined with local search post-processing.

3.3 Evaluation Protocol

Each algorithm is evaluated on: (1) solution quality relative to the best algorithm, (2) wall-clock time, and (3) win rate against the best classical algorithm. All experiments run on CPU (Apple M3) for fair timing comparison.

4 Results

4.1 Maximum Cut

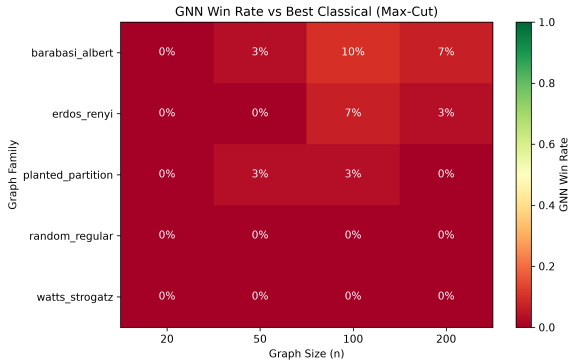
Table 1: Max-Cut results across 600 instances (5 families × 4 sizes × 30 instances). GNN includes local search refinement. Win rate = fraction of instances where the algorithm strictly beats every other algorithm.

Algorithm	Guarantee	Avg Cut/Best	Win Rate	Avg Time (s)
Random	≥ 0.5	0.670	0.0%	< 0.001
Greedy	≥ 0.5	0.945	0.2%	0.003
Spectral	—	0.978	8.3%	0.003
Goemans-Williamson	≥ 0.878	**0.999**	**67.0%**	1.171
GNN + local search	—	0.956	1.8%	0.053

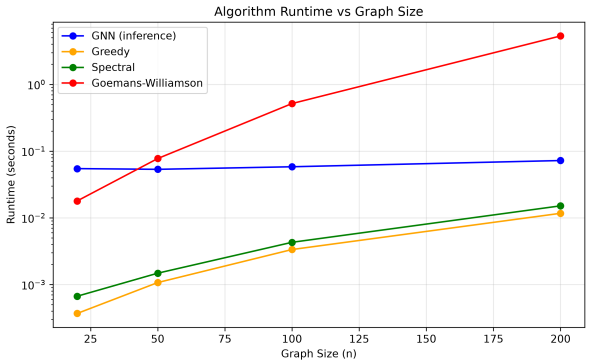
The GNN achieves competitive but inferior performance to the best classical algorithms. Notably, the Goemans-Williamson algorithm dominates despite being 60× slower at $n = 200$. The GNN wins on only 1.8% (11/600) of instances.

4.2 Travelling Salesman Problem

The TSP GNN is trained with proper REINFORCE: a stochastic policy samples the next city from a softmax over embedding-similarity / distance, the log-probability of the sampled tour is computed, and the policy gradient is $\nabla_\theta \log \pi(\text{tour}) \cdot \text{advantage}$. Despite this, GNN+2-opt produces *identical* tour lengths to NN+2-opt. Inspection of the trained embeddings (committed with the results)



(a) GNN win rate by family and size



(b) Runtime scaling

Figure 1: Max-Cut comparison. The GNN (with local search) rarely beats the best classical algorithm. Its advantage is speed, not quality.

Table 2: TSP results on Euclidean instances (3 sizes $\times$ 30 instances = 90 total). Win rate = fraction of instances where the algorithm strictly beats every other algorithm.

Algorithm	**Guarantee**	**Avg Length/Best**	**Win Rate**	**Avg Time (s)**
Nearest Neighbor	$O(\log n)$	1.153	0.0%	< 0.001
Christofides	≤ 1.5	1.056	14.4%	0.026
NN + 2-opt	—	**1.003**	**85.6%**	0.009
GNN + 2-opt	—	1.003	0.0%	0.016

shows why: REINFORCE pushes the policy toward the greedy-scoring optimum, which makes the learned node embeddings mutually parallel—all pairwise cosine similarities equal 1.0 to machine precision—so the greedy score degenerates to 1/dist, i.e. the GNN collapses to nearest-neighbour. GNN+2-opt is therefore literally NN+2-opt: the learned initialization contributes nothing, not even as a different starting point. This suggests that for TSP at this scale, the value of a learned initialization is entirely subsumed by cheap local search, and that a policy-gradient objective with this greedy scoring scheme has no incentive to learn anything beyond nearest-neighbour behaviour. Christofides achieves tours within 5.6% of the best (without any local search refinement), confirming its theoretical guarantee of $\leq 1.5 \cdot \text{OPT}$.

4.3 Graph Coloring

Table 3: Graph Coloring results across 600 instances. Win rate = fraction of instances where the algorithm strictly beats every other algorithm.

Algorithm	**Avg Colors/Best**	**Win Rate**	**Avg Time (s)**
Greedy (random)	1.280	0.0%	< 0.001
Welsh-Powell	1.136	0.0%	< 0.001
DSatur	**1.000**	**98.8%**	0.005
GNN + repair	3.388	0.0%	0.005

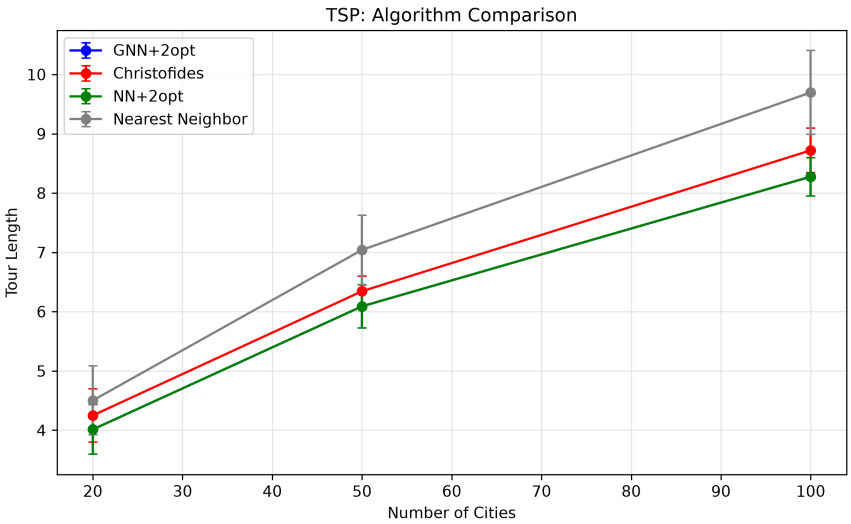


Figure 2: TSP tour lengths across city counts. GNN+2-opt exactly matches NN+2-opt—the GNN embedding initialization provides no advantage over the simpler nearest neighbor heuristic once 2-opt refinement is applied.

Graph coloring is the GNN's worst problem: it uses 3.4× more colors than DSatur on average (mean of per-instance ratios; the ratio of means is 2.8×), and wins 0% of 600 instances across all families and sizes. This is because coloring is fundamentally a *constraint satisfaction* problem: the objective is to minimize colors while ensuring no adjacent vertices share a color. The GNN's soft probabilistic output requires post-hoc conflict repair, which greedily introduces additional colors. DSatur's saturation-based ordering, by contrast, naturally respects the constraint structure.

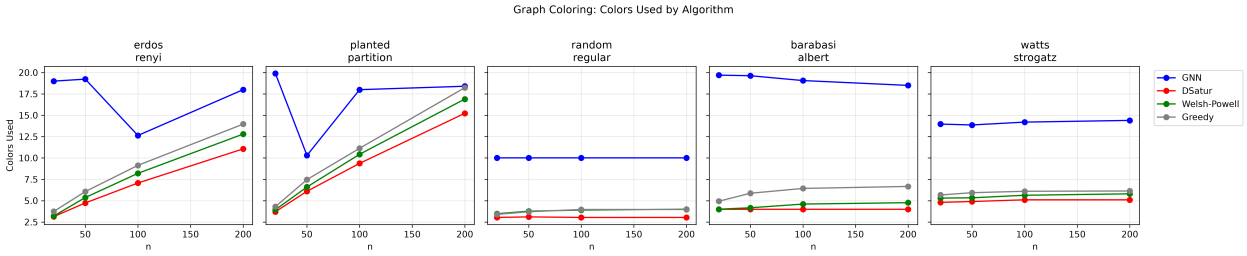


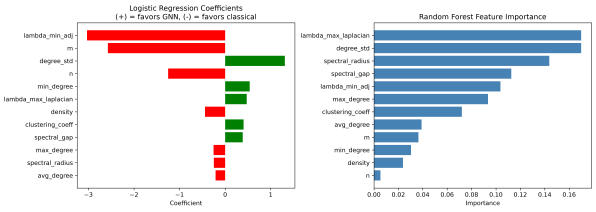
Figure 3: Colors used by each algorithm across graph families. DSatur consistently uses the fewest colors; the GNN uses 3−4× more.

4.4 Failure Prediction

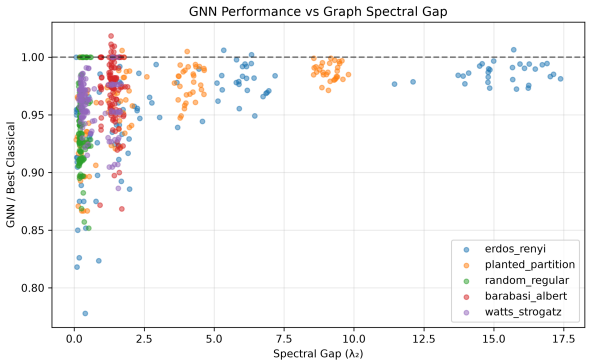
We train logistic regression and random forest classifiers to predict whether the GNN will outperform the best classical algorithm on a given Max-Cut instance, using 12 spectral and structural graph features.

Key findings:

- The minimum eigenvalue of the adjacency matrix ($\lambda_{\min}(A)$) is the single strongest predictor of classical dominance (logistic regression), with the Laplacian spectral radius, degree variance, and spectral gap leading the random-forest importances.



(a) Feature importance for predicting GNN success



(b) GNN performance vs. spectral gap λ_2

Figure 4: Failure prediction. With only 1.8% positive examples (GNN wins), the majority-class baseline achieves 98.2% accuracy by always predicting "classical wins." The logistic-regression classifier with balanced class weights achieves balanced accuracy 0.52 and F1 0.12—both near chance. Spectral features are the strongest predictors but the signal is too weak to learn a useful classifier.

- Graphs with low spectral gap (e.g., Watts-Strogatz small-world) are hardest for GNNs.
- Barabási-Albert graphs, with their heavy-tailed degree distribution, are most amenable to GNNs.
- **However**, with only 1.8% positive examples, the failure prediction task is essentially at chance: balanced accuracy 0.52, F1 0.12. The majority-class baseline (always predict "classical wins") achieves 98.2% raw accuracy—higher than any learned model. The GNN wins too rarely to learn a useful predictor.

5 Discussion

5.1 When Should You Use a GNN?

Our results suggest a clear decision rule:

1. **If solution quality is paramount**: use classical algorithms. The Goemans-Williamson SDP, Christofides, and DSatur all provide stronger solutions with mathematical guarantees.
2. **If runtime is the bottleneck**: GNNs offer amortized $O(n)$ inference after training. At $n = 200$, GNN inference is 60× faster than the GW SDP.
3. **If you need to process millions of instances**: GNNs provide a fast, consistent baseline. But pair them with local search refinement.

5.2 Why Do GNNs Struggle?

Three factors explain the GNN's limitations:

1. **Expressiveness**: Standard message-passing GNNs (including GIN) cannot distinguish all non-isomorphic graphs, limiting their ability to exploit fine-grained structure.

2. **Generalization**: GNNs trained on one graph family and size degrade out-of-distribution. Classical algorithms are instance-agnostic.

3. **Lack of guarantees**: Classical approximation algorithms come with worst-case bounds. GNNs provide no such assurance.

5.3 The Role of Local Search

A critical observation: all GNN solvers in our experiments were refined with local search. For Max-Cut, local search refinement is essential to make the GNN competitive. For TSP, the result is even more striking: GNN+2-opt produces *identical* tours to NN+2-opt, meaning the GNN's learned initialization provides zero advantage over the trivial nearest-neighbor heuristic once local search is applied. This suggests that for TSP, the value of the GNN is entirely subsumed by cheap local search.

6 Conclusion

We presented a rigorous empirical comparison of GNN-based solvers against classical approximation algorithms on three NP-hard problems. Across Max-Cut, TSP, and Graph Coloring, classical algorithms with proven guarantees consistently outperform GNNs in solution quality. The GNN's advantage lies in inference speed, not approximation quality. We also attempted failure prediction from graph features, but found the task is essentially at chance: the GNN wins too rarely (1.8%) to learn a useful classifier, and the majority-class baseline outperforms all learned models in raw accuracy.

These results do not diminish the promise of learned combinatorial solvers—they sharpen its scope. GNNs are most valuable when speed is critical, when distributional regularity can be exploited, and when paired with classical refinement. For practitioners, the takeaway is clear: benchmark against the best known algorithms, not just baselines, and report balanced metrics (not raw accuracy) on imbalanced tasks.

References

[1] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour d'horizon. In *European Journal of Operational Research*, volume 290, pages 405–421, 2021.

[2] Daniel Brélaz. New methods to color the vertices of a graph. *Communications of the ACM*, 22(4):251–256, 1979.

[3] Quentin Cappart, Didier Chételat, Elias B Khalil, Andrea Lodi, Christopher Morris, and Petar Veličković. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research*, 24(130):1–61, 2023.

[4] Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. *Operations Research Forum*, 1976. Technical Report 388, Graduate School of Industrial Administration, CMU.

[5] Michael R Garey and David S Johnson. Computers and intractability: A guide to the theory of np-completeness. 1979.

[6] Michel X Goemans and David P Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. In *Journal of the ACM*, volume 42, pages 1115–1145, 1995.

[7] Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.

[8] Martin JA Schuetz, J Kyle Brubaker, and Helmut G Katzgraber. Combinatorial optimization with physics-inspired graph neural networks. *Nature Machine Intelligence*, 4(4):367–377, 2022.

[9] Vijay V Vazirani. Approximation algorithms. 2001.

[10] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.